```
class PhoneBook
  asObject = (row) ->
    [name, number, relationship] = row.split ','
    { name, number, relationship }

  asString = (data) ->
    "#{data.name}: #{data.number} (#{data.relationship})"

  print = (s) ->
    s.join '\n'

  relationshipIs = (relationship) ->
    (data) -> data.relationship is relationship

  nameIs = (name) ->
    (data) -> data.name is name

  constructor: (sourceCsv) ->
    csv = new CSVRowEmitter sourceCsv
    @numbers = withEvents(csv, 'row')

  list: (relationship) ->
    evaluated = \
    if relationship
      @numbers
      .map(asObject)
      .filter(relationshipIs relationship)
      .evaluate()
    else
      @numbers
      .map(asObject)
      .evaluate()

    print(asString data for data in evaluated)

  get: (name) ->
    evaluated = \
    @numbers
    .map(asObject)
    .filter(nameIs name)
    .evaluate()

    print(asString data for data in evaluated)

console.log "Phonebook. Commands are get, list and exit."

process.stdin.setEncoding 'utf8'
stdin = process.openStdin()

phonebook = new PhoneBook 'phone_numbers.csv'
stdin.on 'data', (chunk) ->
  args = chunk.split ' '
  command = args[0].trim()
  name = relationship = args[1].trim() if args[1]
  console.log switch command
    when 'get'
      phonebook.get name
    when 'list'
      phonebook.list relationship
    when 'exit'
      process.exit 1
    else 'Unknown command'
```

Things that operate on a phone book belong in the PhoneBook class.

Use the fluent interface created.

Read input from the console.